

Documentation technique - Fraya Mobile

Date de generation : 22 septembre 2026

1. Vue d'ensemble

Fraya Mobile est une application Flutter de VTC pour Abidjan, Cote d'Ivoire. Le meme code source produit plusieurs variantes de l'application grace a un systeme de flavors :

- passenger : application passager, lancee par lib/main_passenger.dart.
- driver : application chauffeur, lancee par lib/main_driver.dart.
- dev : variante de developpement, lancee par lib/main_dev.dart.

Le widget racine est FrayaApp dans lib/app.dart. Il configure le theme, la localisation, le router, les listeners globaux, les notifications push, le suivi de session et les actions au retour d'arriere-plan.

2. Technologies principales

Domaine	Technologie
Framework mobile	Flutter
Langage	Dart
Gestion d'etat	Riverpod 3, riverpod_generator
Navigation	go_router
HTTP	Dio
Temps reel	socket_io_client
Cartographie	google_maps_flutter, Google Directions/Places
Notifications	OneSignal, Firebase
Crash reporting	Firebase Crashlytics
Stockage local	shared_preferences, flutter_secure_storage
Documents et medias	image_picker, file_picker, pdf, pdfx, flutter_image_compress
Tests	flutter_test

Version applicative declaree dans pubspec.yaml : 1.0.2+6.

3. Architecture du code

Le projet suit une organisation proche de Clean Architecture :

```
lib/  
  app.dart  
  main_dev.dart  
  main_driver.dart  
  main_passenger.dart  
  core/  
    bootstrap/  
    config/  
    error/  
    models/  
    realtime/  
    router/  
    services/  
    theme/  
    utils/  
  data/  
    repositories/  
    sources/  
      local/  
      remote/  
  domain/  
    models/  
    repositories/  
    usecases/  
  features/  
    driver/  
    passenger/  
  shared/  
    models/  
    providers/  
    screens/  
    widgets/
```

Responsabilites par couche

- core : configuration globale, bootstrap, theme, routage, services transverses, erreurs, constantes et outils.
- data : clients reseau, sources locales/distantes et implementations des repositories.
- domain : contrats de repositories, modeles metier et cas d'utilisation.
- features : ecrans, providers et widgets organises par domaine fonctionnel.
- shared : composants reutilisables, providers globaux et ecrans communs.

4. Bootstrap et configuration

Le lancement passe par `runFrayaApp(AppFlavor flavor)` dans `lib/core/bootstrap/app_bootstrap.dart`.

Sequence d'initialisation :

1. Initialisation Flutter avec `WidgetsFlutterBinding.ensureInitialized()`.
2. Chargement du fichier `.env`.
3. Initialisation de `AppConfig` selon le flavor.

4. Initialisation du logger.
5. Initialisation du stockage local.
6. Initialisation du client API Dio.
7. Lancement de FrayaApp dans un ProviderScope.
8. Initialisation differee des notifications push et de Crashlytics.

AppConfig definit le nom de l'application, l'URL API et le niveau de logs :

Flavor	Nom	Base URL	Logs
passenger	Fraya Taxi	production	non
driver	Fraya Chauffeur	production	non
dev	Fraya Dev	developpement	oui

Les variables attendues dans .env sont documentees par .env.example :

```
GOOGLE_MAPS_API_KEY_ANDROID=  
GOOGLE_MAPS_API_KEY_IOS=  
GOOGLE_PLACES_API_KEY=  
SERVER_URL=  
ONESIGNAL_APP_ID_PASSENGER=  
ONESIGNAL_APP_ID_DRIVER=  
WHATSAPP_SUPPORT_NUMBER=
```

5. Routage

Le router est fourni par appRouterProvider dans lib/shared/providers/app_providers.dart. Il choisit automatiquement le router passager ou chauffeur selon AppConfig.instance.

Routes passager

Chemin	Ecran
/	Splash / skeleton
/onboarding	Onboarding passager
/login	Connexion passager
/register	Inscription passager
/passenger/home	Accueil passager
/passenger/vehicle	Selection / suivi de course
/passenger/tracking	Suivi de course
/passenger/complete	Resume et notation de course
/history	Historique des courses
/profile	Profil passager
/passenger/ride-details	Detail d'une course
/passenger/favorites	Adresses favorites
/passenger/promotions	Promotions
/settings	Parametres passager
/forgot-password	Mot de passe oublie
/passenger/wrong-account	Mauvais type de compte

Le redirect passager tient compte de l'etat d'authentification, de l'onboarding, d'une course active et du nettoyage d'une recherche chauffeur en attente.

Routes chauffeur

Chemin	Ecran
/	Splash / skeleton
/login	Connexion chauffeur
/register	Inscription chauffeur
/driver/home	Accueil chauffeur
/driver/earnings	Revenus
/driver/history	Historique
/driver/wallet	Portefeuille
/driver/profile	Profil chauffeur
/driver/settings	Parametres chauffeur
/driver/status	Fin de course
/driver/kyc	KYC chauffeur
/driver/vehicle	Vehicule
/driver/vehicle/pending	Vehicule en attente
/forgot-password	Mot de passe oublie
/driver/wrong-account	Mauvais type de compte

Le redirect chauffeur applique les regles d'accès selon l'authentification, le statut administrateur, le KYC et l'état du véhicule.

6. Fonctionnalités métier

Parcours passager

- Authentification et inscription en plusieurs étapes.
- Onboarding et persistance de l'état d'onboarding.
- Recherche d'itinéraire, estimation tarifaire et choix de catégorie.
- Demande de course, recherche chauffeur et restauration de session.
- Suivi de course active, partage de suivi et SOS.
- Annulation, notation, pourboire et signalement de problème.
- Historique, détails de course et reçu.
- Profil, photo, changement de téléphone et suppression de compte.
- Favoris, promotions, paramètres et support WhatsApp.

Parcours chauffeur

- Authentification et inscription chauffeur.
- Soumission KYC et gestion documentaire.
- Ajout et mise a jour du vehicule.
- Presence chauffeur et service foreground Android.
- Recuperation des courses disponibles.
- Acceptation, arrivee, demarrage, finalisation et annulation de course.
- Localisation live et heartbeat.
- Historique, revenus, profil, parametres et portefeuille.
- Rechargement, retrait et souscription a des packages.

7. API et reseau

Le client HTTP central est ApiClient dans lib/data/sources/api_client.dart.

Caracteristiques :

- Base URL definie par AppConfig.
- Timeouts de 30 secondes.
- Headers JSON par default.
- Injection automatique du token Bearer depuis le stockage securise.
- Refresh token automatique sur erreur 401 hors endpoints d'authentification.
- Notification globale d'expiration de session.
- Mapping des erreurs Dio vers exceptions metier.
- Logging HTTP active uniquement en flavor dev.

Endpoints principaux utilises

Domaine	Endpoints
Auth	/auth/login, /auth/register, /auth/register/step3, /auth/refresh-token, /auth/forgot-password, /auth/reset-password
Profil	/sid-users/profile, /sid-users/delete/{userId}
Prix	/pricing/configs/range-pricing, /rides/maps/estimate
Courses passager	/rides/maps/request, /rides/maps, /rides/maps/user/{userId}/active, /rides/maps/{rideId}/cancel, /rides/maps/{rideId}/rate, /rides/maps/{rideId}/share
Chauffeurs proches	/rides/maps/drivers/nearby
Courses chauffeur	/rides/maps/available, /rides/maps/driver/{driverId}/active, /rides/maps/{rideId}/accept, /arrived, /start, /complete, /cancel
KYC	/kycs, /kycs/sid-user/{sidUserId}
Vehicule	endpoints de creation et mise a jour vehicule via driver_vehicle_remote_data_source.dart
Notifications	/notifications/own
Favoris	/favorites, /favorites/create, /favorites/{id}
Promotions	/promotions/apply
Wallet chauffeur	/subscriptions/reloads, /packages
Presence chauffeur	/sid-users/me/heartbeat

8. Stockage local et securite

Le stockage local est gere par LocalStorage avec :

- flutter_secure_storage pour les tokens et donnees sensibles.
- shared_preferences pour les preferences et etats simples.

Cles importantes dans AppConstants :

Cle	Usage
access_token	Token passager
driver_access_token	Token chauffeur
refresh_token	Token de refresh
auth_user_data	Donnees utilisateur passager
driver_auth_user_data	Donnees utilisateur chauffeur
onboarding_complete	Etat d'onboarding
active_ride_id	Course active
pending_search_ride_id	Recherche chauffeur en attente
passenger_booking_session	Session de reservation
passenger_register_draft	Brouillon inscription passager
driver_register_draft	Brouillon inscription chauffeur
driver_onboarding_draft	Brouillon onboarding chauffeur

9. Temps reel, notifications et cycle de vie

L'application utilise :

- socket_io_client pour les evenements temps reel.
- OneSignal pour les notifications push.
- Firebase Crashlytics pour les erreurs fatales et Flutter.
- Un coordinateur de reprise d'application pour rafraichir les donnees apres un long passage en arriere-plan.

Sur le flavor chauffeur, l'application active aussi la logique de presence chauffeur :

- driverPresenceControllerProvider.
- DriverPresenceForegroundService sur Android.
- Synchronisation a la fermeture, au detach et au resume de l'application.

10. Permissions mobiles

Android

Permissions declarees dans android/app/src/main/AndroidManifest.xml :

- Internet et etat reseau.
- Localisation fine et approximative.
- Camera.
- Notifications.

- Foreground service.
- Wake lock et boot completed.
- Reception FCM.
- Lecture stockage externe jusqu'a Android 12.

Le manifeste declare aussi :

- DriverPresenceForegroundService.
- Channel notification Firebase fraya_alerts_v2.
- Cle Google Maps Android dans les meta-data.
- Queries pour Google Maps, telephone, HTTPS, navigation et WhatsApp.

iOS

Permissions principales dans ios/Runner/Info.plist :

- Localisation en usage et toujours.
- Camera.
- Phototheque.
- Schemes Google Maps et WhatsApp.

11. Assets et configuration Flutter

Assets declares dans pubspec.yaml :

- .env
- assets/config/
- assets/images/
- assets/icons/
- assets/fonts/
- assets/songs/

Le projet configure aussi :

- flutter_launcher_icons pour Android/iOS.
- flutter_native_splash avec fond noir et image de splash Fraya.

12. Tests

Le dossier test/ contient des tests unitaires et widgets couvrant notamment :

- Routage passager et chauffeur.
- Providers Riverpod.
- Repositories data.
- Sources remote et payload builders.
- Services de localisation, geocoding, notifications, partage et navigation.
- Modeles metier.
- Widgets d'historique, booking, profil, vehicule et accueil chauffeur.

Commandes utiles :

```
flutter test  
flutter analyze
```

13. Build et execution

Commandes typiques :

```
flutter pub get  
flutter run -t lib/main_passenger.dart  
flutter run -t lib/main_driver.dart  
flutter run -t lib/main_dev.dart
```

Build Android :

```
flutter build apk -t lib/main_passenger.dart --release  
flutter build apk -t lib/main_driver.dart --release
```

Build iOS :

```
flutter build ios -t lib/main_passenger.dart --release  
flutter build ios -t lib/main_driver.dart --release
```

Un script PowerShell scripts/build_release.ps1 est present pour aider a produire des builds release.

14. Points d'attention techniques

- La cle Google Maps Android est actuellement visible dans AndroidManifest.xml; idealement, elle devrait etre geree via secrets ou variables de build.
- Le fichier .env est declare comme asset Flutter : verifier qu'il ne contient aucune information sensible avant distribution.
- Le refresh token est centralise dans ApiClient; toute nouvelle source remote doit passer par ApiClient.instance.dio.
- Les routes chauffeur dependent fortement de DriverAdminStatusGate; toute evolution des statuts backend doit etre repercutee dans ce resolver.
- Les donnees API sont parfois unwrappees avec plusieurs formats possibles (data, result, listes imbriquees). Maintenir cette tolerance si le backend n'est pas strictement stable.
- Les tests existants doivent etre conserves lors des modifications de providers, routes et payload builders, car ils couvrent les regressions les plus probables.

15. Glossaire

Terme	Definition
Flavor	Variante de l'application lancee avec une configuration specifique
KYC	Verification d'identite et de documents chauffeur
Provider	Unite Riverpod exposant un etat ou une dependance
Repository	Contrat ou implementation d'accès aux donnees metier
Data source	Acces concret a une source locale ou distante
Use case	Action metier isolee dans la couche domain
Ride	Course VTC
Foreground service	Service Android visible permettant de maintenir une presence active